Computer Architecture - 1DT093

ExamTag: 2024-May
Exam Date: 2024-05-24

Instructions

Format: This exam consists of 6 multiple choice questions for each of the 12 lectures in the course. You may have to do some calculations in order to determine the correct answer. The motivation for having so many questions is to get a more accurate assessment of your knowledge of the material.

Grading: To discourage guessing, each incorrect answer will be worth -1/3 point, while each correct answer is worth +1 point. If you don't answer you will get 0 points for that question. I.e., if you guess randomly, the expected score is 0.

Material: You are allowed a calculator and one double-sided, hand-written A4 sheet of notes during the exam.

Good luck!

Note: Please take this exam seriously and pass it! (Exams take huge amounts of teaching time to produce and grade, and most students and teachers would prefer that the time be spent teaching instead.)

1. ISA 1

1. In MIPS assembly, what special meaning does the "exit:" label have compared to other labels?
    1. It has no special meaning
    2. It is used in branch instructions to jump to the end of a loop
    3. It is used in branch instructions to jump to the end of the program
    4. It marks the end of the current function
2. Why do processors do all their computations on registers?
    1. Main memory is too slow
    2. Fewer bits are needed in the instruction to access registers compared to memory locations
    3. A and B
    4. None of the above
3. If we extended MIPS to have each register hold 128-bits, how many bytes would be needed to store all of the data of the register file?
    1. 496
    2. 512
    3. 3,968
    4. 4,096
4. What instructions go in XXX and YYY to accomplish:
```
if (R3==R4) then
  R5=2
else
  R5=1?

XXX R3, R4, labelA
addi R5, R0, 1
YYY R0, R0, labelB
labelA:
addi R5, R0, 2
labelB:
```
    1. XXX=bne YYY=j
    2. XXX=beq YYY=j
    3. XXX=bne YYY=beq
    4. XXX=beq YYY=beq
5. How many addressable registers can be used as operands or destinations for the `add` instruction?
    1. 31
    2. 32
    3. 33
    4. More than 33
6. From where do immediate instructions get their immediate values?
    1. The register file
    2. The instruction
    3. The ALU
    4. Depends on the instruction type

2. ISA 2

1. Why does the PC (program counters) always need to end with 00?
    1. The last to bits of the instructions are always left empty.
    2. All instructions are word-aligned.
    3. All instructions in MIPS are 8 byte aligned, and thus the last two bits are zeros.
    4. All of the above
2. How does the processor know the instruction type?
    1. The processor checks if the instruction has 0, 2, or 3 register fields
    2. The processor checks if there is an immediate in the instruction
    3. The checks the opcode bits region of each instruction
    4. None of the above
3. We used the lui (load upper immediate) and the ori (or immediate) instructions to load a full 32-bit constant. What can you say about the sign extension of the immediate field?
    1. All immediates are sign-extended
    2. At least the ori instruction must not sign-extend its immediate
    3. Can't conclude anything since sign-extending as part of an OR won't matter
    4. This code example is too simple to tell us anything
4. In the MIPS arechitecture we studied, how would you conditionally jump 1 million instructions forward in a program?
    1. Use a standard bne/beq instruction
    2. Use a standard j instruction
    3. Use a bne/beq instruction to control whether you execute a j instruction
    4. You can't because conditional branches only have one constant
5. Would the `j 0x340` instruction behave in the same way as a `beq R1, R1, 0x340` instruction? (For simplicity don't think about pipelined processors yet)
    1. Yes, both instructions would branch/jump unconditionally
    2. Maybe, depends on the value in register R1
    3. No, the immediate will not fit in the `beq`>instruction, hint: 0x340 = 0b 11 0100 0000 (in binary)
    4. No, the two instructions will branch/jump to different places.

6. What is the value of $ra right after instructions 8, 24, and 34?

```
0:  addi R1, $zero, 5
4:  addi R2, $zero, 7
8:  jal  process
12: j    exit
16: process:
     add  R3, R1, R2
20:  sub  R4, R1, R2
24:  jal  add
28:  jr   $ra
32: add:
     addi R3, R1, 2
34:  jr   $ra
```

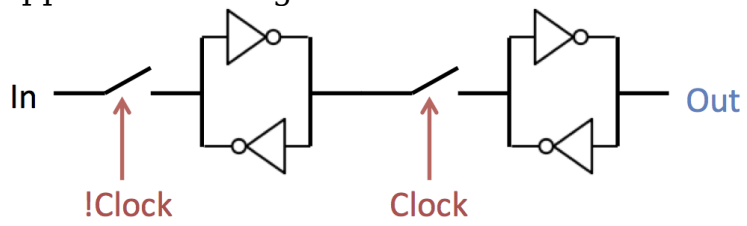
   1. 8, 24, 34
   2. 8, 24, 24
   3. 12, 28, 12
   4. 12, 28, 28

3. Computer Arithmetic

1. Why do computers use binary numbers instead of tertiary (3 levels) or analog (infinite levels) signals?
    1. Can't design non-binary circuits
    2. Can integrate lots of circuits without noise adding up
    3. A and B
    4. None of the above
2. Without normalization, how many ways are there to represent the number 3 with an `seeeffff` floating point format? (-1^{S}x0.ffffx$2^{(eee-4)}$)
    1. 1
    2. 2
    3. 3
    4. 4
3. What would the following 2's compliment math compute: A+!B
    1. A-B
    2. A+B-1
    3. A-B-1
    4. Garbage
4. What is the result of (-4)+(-6) for 4 bit two's complement numbers?
    1. 10111
    2. 10110
    3. 0111
    4. Overflow
5. What is the result of the following signed-magnitude addition: 1000+0001?
    1. -1
    2. 0
    3. 1
    4. 8
6. For an 8-bit, signed-magnitude, fixed-point number format `#####.###`, what is the largest positive value you can represent?
    1. 7.5
    2. 15.875
    3. 31.875
    4. 120

4. Logic

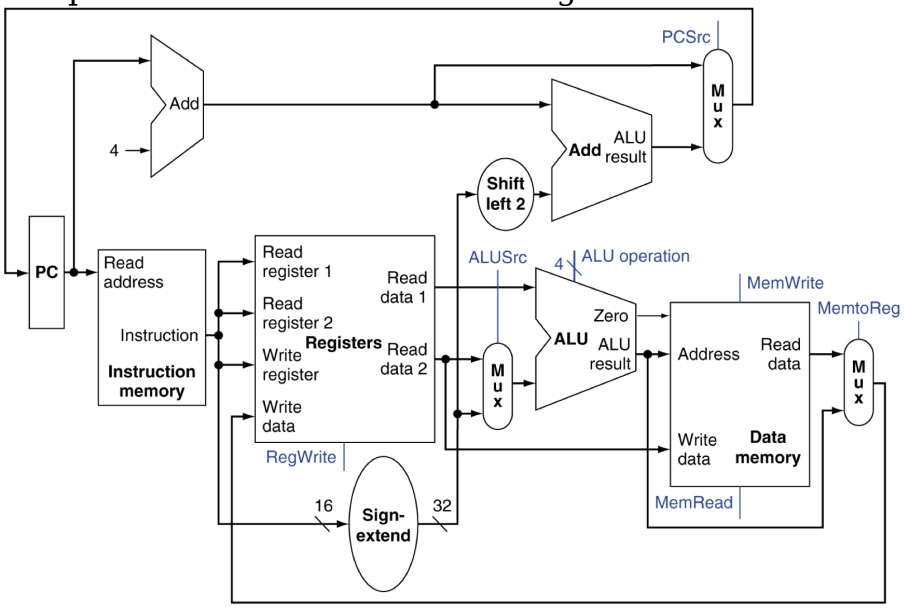
1. Which statement about DRAM and SRAM is true?
    1. DRAM keeps its values in a feedback loop.
    2. SRAM is 6-10x smaller than DRAM.
    3. SRAM is faster than DRAM.
    4. All of the above
2. (!A AND B) OR (!B AND A) is the equivalent of which of the following?
    1. A AND B?
    2. A OR B
    3. A XOR B?
    4. A XNOR B
3. How many inputs will a 4-bit mux with a 3-bit select signal have?
    1. 3 inputs each 4 bits wide
    2. 4 inputs each 3 bits wide
    3. 8 inputs each 4 bits wide
    4. 16 inputs each 3 bits wide
4. What will be the value of R12 after executing the following two instructions?
   and R12, R12, R12
   or R12, R12, R0
    1. 0x0000 (Zero)
    2. 0xFFFF (All ones)
    3. It depends on the value of R12 before the first instruction
    4. The value of R12 will not change
5. For a incrementing counter logic (counter value increases every cycle), what does the combinational part of the logic do?
    1. Store the current value
    2. Store the next value
    3. Calculate the current value
    4. Calculate the next value
6. What changes from a simple latch when you have two latches back to back with opposite clock signals?



    1. The latch behaves the same but it takes longer for the data to get through
    2. The data is only updated when the clock goes from low to high
    3. The data only goes through the latch when the clock is low
    4. The latch can store a value when the clock is low

5. Processor Control and Datapath

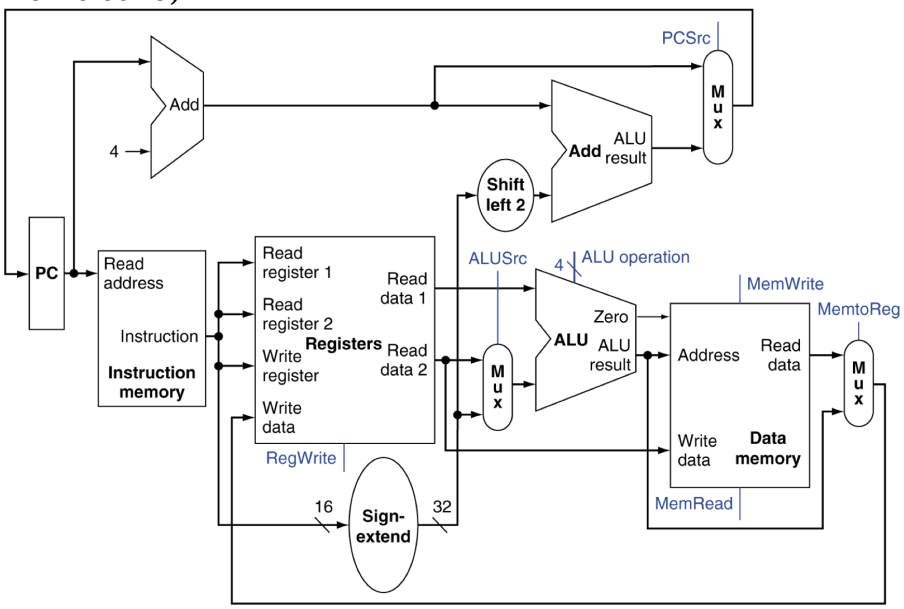
1. What is the shortest clock cycle time for the pipeline below if the instruction memory takes 5ns, the registers take 1ns to read or write, sign-extension takes 0.5ns, the ALU takes 2ns, the data memory takes 10ns, and the branch computation takes 2ns after the sign extension?



   1. 8ns
   2. 18ns
   3. 19ns
   4. 21ns

2. JAL is a J format instruction. How does the datapath know which register to store the return address into?
   1. The instruction specifies the destination register
   2. The immediate field specifies the destination for storing the return address
   3. The return address is automatically put in the PC register
   4. The value is generated by the control logic based on the opcode

3. What are RegWrite and MemtoReg for a sw instruction? (Assume MemtoReg=1 comes from the Data memory and MemtoReg=0 comes from the ALU. X means Don't care)



   1. RegWrite = 1, MemtoReg = 1
   2. RegWrite = 1, MemtoReg = 0 (can't read while we write, so we have to choose 0)
   3. RegWrite = 0, MemtoReg = X
   4. RegWrite = 0, MemtoReg = 0 (can't read while we write, so we have to choose 0)

4. What is the ALU operation for a sw instruction?
   1. Add
   2. Subtract
   3. Pass through
   4. Undefined (ALU results not used)

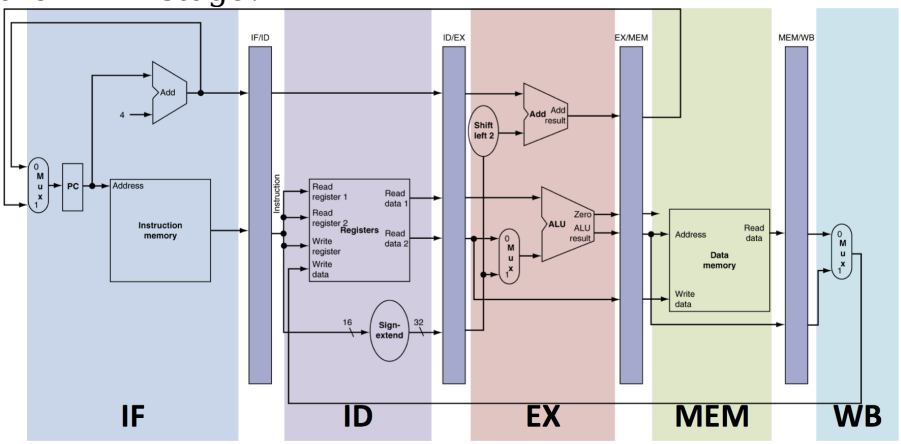
5. What is the output of the sign extender for non I-format instructions?
   1. Zero: non-I-format instructions do not have an immediate to sign-extend
   2. Nothing: non-I-format instructions do not have an immediate so the sign-extension logic is turned off
   3. Garbage: non-I-format instructions do not have an immediate so the value is whatever bits were in that part of the instruction
   4. Next immediate: the sign-extender looks ahead to the next instructions immediate to get a head start in extending the sign

6. Why do we need two outputs from the control logic to properly handle bne and beq?
   1. We need one for branch taken and one for not-taken
   2. We need one to signal that this is a branch instruciton, another to control whether to branch forwards or backwards.
   3. We need two to determine how to use the Zero output for the two branches
   4. We don't: we can do the logic with a single branch output from the control

6. Pipelining

1. The 5-stage pipelined processor (as we learned in class) take the following amount of time to complete each stage (15ns, 10ns, 20ns, 15ns, 10ns) respectively. What is the maximum frequency of this particular design (denoted `baseline` in the options below)? How much quicker would the processor run (frequency) if the fetch and memory stages could be reduced by 5ns each (denoted `New` in the options)?
   Hint: Frequency = 1/Cycle time, 1/1ns = 1GHz, 1/10ns = 100MHz, etc.
   1. Baseline: 100 MHz (1/10ns), New: 100 MHz (1/10ns)
   2. Baseline: 66 MHz (1/15ns), New: 50 MHz (1/20ns)
   3. Baseline: 66 MHz (1/15ns), New: 100 MHz (1/10ns)
   4. Baseline: 50 MHz (1/20ns), New: 50 MHz (1/20ns)
2. What is true about pipelining?
   1. Pipelining will result in shorter latency of executing each instruction
   2. Pipelining will not affect the latency of executing each instruction
   3. Pipelining will result in higher instructions throughput
   4. Pipelining will increase the clock cycle and thus lower the frequency of the processor compared to a single-cycle processor
3. A single-cycle processor takes 100ns per instruction. It is perfectly pipelined into 10 stages with pipeline registers that take 2ns and can be kept 60% full. What change in instruction throughput can you expect compared to the single?cycle processor?
   1. 60% slowdow
   2. 60% speedup
   3. 5x speedup
   4. 10x speedup
4. A single-cycle processor takes 100ns and uses 100pJ. It is perfectly pipelined to 20 stages (20 registers), with pipeline registers that take 2ns and 2pJ in energy. What is the energy per instruction for the pipelined processor as a percentage of the original one?
   1. 20%
   2. 40%
   3. 100%
   4. 140%

5. Why do we need to store Register File output 2's data in the pipeline register for the MEM stage?



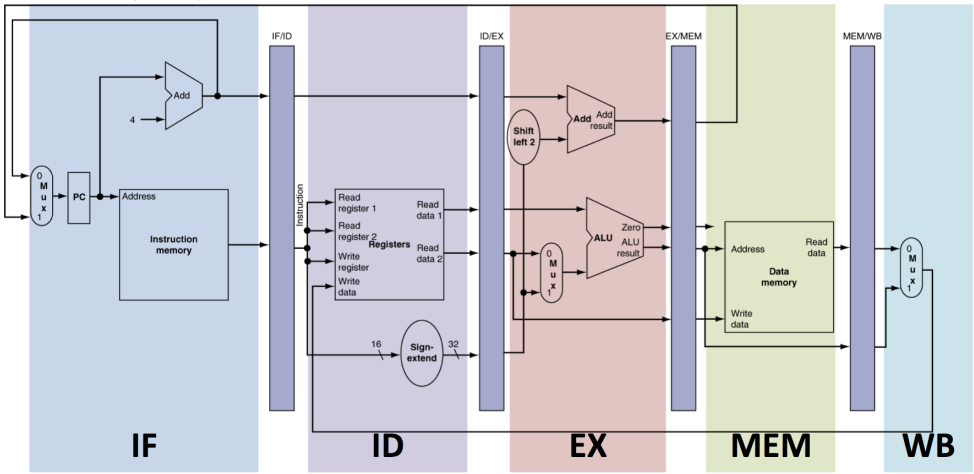
1. Needed for the store instruction's data
2. Needed for the store instruction's address
3. Needed for writeback later in the WB stage
4. We don't need to store it

6. How many branch delay slots are needed in the pipeline shown above?
   1. 0
   2. 1
   3. 2
   4. 3

7. Hazards

1. Choose the option below with the correct answer about the promises the MIPS ISA makes and how pipelining may affect it.
    1. Modular Execution: All executions need to be done in a modular manner, thanks to pipelining, an instruction will only be executing in one stage at a time.
    2. Sequential execution: This promise is broken as multiple instructions will be running at the same time (in parallel)
    3. Atomic execution: This promise is kept as all instructions only make meaningful changes at the final writeback stage
    4. Modular Execution: all stages are executed at the same time, breaking the promise of modularity.
2. What would happen to performance if a pipeline could only read or write to the register file on each cycle?
    1. No change
    2. Better performance because the register file would be simpler and faster
    3. Slightly worse performance because `lw` happens more frequently than `sw`
    4. Terrible performance because most instructions would have to stall
3. A processor is ideally pipelined to 10 stages, but due to hazards, 25% of the instructions are NOPs. What is the speedup over the original processor?
    1. 10x
    2. 2.5x
    3. 25x
    4. 7.5
4. What is needed to resolve any hazards in this example?
```
add R14, R1, R13
lw R14, 4(R2)
sw R14, 4(R13)
addi R2, R13, R1
```
    1. Forwarding from MEM to EX
    2. Forwarding from WB to MEM
    3. Can't resolve; need to stall
    4. There are no hazards to resolve
5. Why can't we forward to avoid the branch delay slot?
    1. We can if we add a ID to IF forwarding path
    2. We can't forward the PC, only register values
    3. We can because data is ready but in the wrong place, thus forwarding can solve the control hazard.
    4. Branch delay slots solve a different problem to the problem forwarding solves

6. What is needed to resolve the following hazard?
   lw R4, 12(R14)
   add R14, R4, R13



1. Forwarding from WB to EX
2. Forwarding from MEM to EX
3. Double-pumping the RF
4. Can't resolve: need to stall

8. Branch Prediction and Exceptions and Interrupts

1. What is the slowdown for processor that do one instruction per cycle and has a branch predictor that is 70% accurate with a 30-cycle misprediction cost? 40% of instructions are branches.
    1. ~4.5x slower
    2. ~2.5x slower
    3. ~1.5x faster
    4. More information needed
2. How many branches are correctly predicted with a 2-bit predictor initialized to strongly not taken? (hint `0 % 2 == 0`

    ```
    for (i=0; i<100,000; i++) {
            if (i%2==1) { // branch 1: if i is odd
                    // do something
            }
    } // branch 2: loop
    ```

    python:
    ```
    for i in range(0, 100000):
            if i % 2 == 1: # branch 1: if i is odd
                    # Do something
            # branch 2: loop
    ```

    1. branch 1: 50,000/100,000 branch 2: 99,997/100,000
    2. branch 1: 75,000/100,000 branch 2: 50,000/100,000
    3. branch 1: 50,000/100,000 branch 2: 99,998/100,000
    4. branch 1: 99,998/100,000 branch 2: 99,997/100,000
3. A processor's branch predictor has 100 entries. What happens for a long-running program that has 200 branches, but only uses 100 for the first half of the program and the other 100 in the second half?
    1. Half of the branches will not get predictions
    2. The processor will crash
    3. The branches will mess up each other's predictions
    4. The predictor will work well for all branches
4. How do pipelines complicate exceptions?
    1. They don't: exceptions and interrupts just cause the program to jump to another piece of code
    2. They don't: Even for precise exceptions, as soon as an exception is triggered anywhere in the pipeline, the processor must immediately transition into the OS to handle the excception.
    3. Exceptions from later instructions may come before exceptions from earlier instructions depending on where they are in the pipeline
    4. B and C

5. Why is BTFN (backwards taken, forwards not-taken) a reasonable idea for a predictor?
    1. It isn't
    2. Loops jump backwards and you mostly take those branches
    3. Loops jump forwards and you mostly don't take those branches
    4. Error checks are usually taken
6. Which statement is false?
    1. Static predictors can adapt to program behavior
    2. Dynamic predictors rely on the compiler to know what branches will do
    3. Dynamic predictors are always better than static predictors
    4. All of the above

9. Input/Output

1. What is the correct about memory-mapped I/O (MMIO)?
    1. Memory mapped I/O allows using normal load and store instructions to specific physical address locations to communicate with I/O devices.
    2. MMIO enables the CPU to do other work while the MMIO transfers the data between devices.
    3. MMIO should be used with interrupts instead of polling because this helps with lower-latency I/O.
    4. MMIO is set up using direct memory access (DMA), and MMIO is used to move data between I/O devices for the CPU.
2. What is correct about Direct memory access (DMA)
    1. DMA speeds up LD/ST from/to I/O devices because it skips the cache and goes directly to memory.
    2. DMA enables using normal load or store instructions to access the IO devices
    3. DMA cannot be set up with interrupts and can only work with polling
    4. DMA allows the CPU to do other work while DMA takes care of transferring data across I/O devices.
3. What are the tradeoffs between polling and interrupts?
    1. Polling uses up little processor time but has a high latency
    2. Interrupts use up a lot of processor time but have a low latency
    3. Interrupts are good for low-latency I/O and polling for high-throughput I/O
    4. Interrupts use up little processor time but polling gives low latency
4. Why have busses become a problem in modern systems?
    1. Hard to make small wires
    2. Hard to connect many wires
    3. Hard to find space for long wires
    4. Hard to keep the data on the wires synchronized
5. What is true in the options below
    1. NVMe protocol was introduced to help speedup spinning hard disk drives (HDD).
    2. Solid state drives (SSDs) connected through SATA are no better than spinning HDDs in terms of random accesses
    3. With shorter latency I/O devices, it is more advantageous to do polling instead of interrupts for lower-latency I/O.
    4. Spinning hard disks are much better at performing random I/O than sequential I/O

6. What is true about the options below
    1. All data movement in the computer must go through the main memory (main DRAM), e.g. data from disk that needs to go to the GPU must go through the DRAM.
    2. Compressing data to reduce transfer is a bad idea because of the overheads of compressing/decompressing and is not used in practice.
    3. Pipelining data transfer with computation is a one effective strategy to reduce data transfer/computation latency.
    4. The higher the transfer bandwidth of the channel, the more it will benefit from compressing the data.

10. Caches

1. Why would we build a set-associative cache?
    1. It's simpler than direct mapped
    2. It handles conflict misses better than fully-associative
    3. It has a smaller cache line size
    4. It has some associativity but isn't too complex
2. Why do we want caches?
    1. DRAM is slow
    2. Processors are fast
    3. Lots of instructions access memory
    4. All of the above
3. In the following code, what is the miss ratio of the code below on a 8KB fully-associative LRU cache with a 64 bytes cache line size?

    ```
    int array[4096] //each int is a word (or 4 bytes)
    for (int i = 0; i < 4096; ++i)
      array[i] = array[i] + 2;  // one read, one write
    ```

    1. Around 100%
    2. Around 6%
    3. Around 3%
    4. Around 0%
4. Assume you have a 2-way set-associative LRU cache with 4 entries and a cache line size of 1 bytes. What will the hit (H) miss (M) pattern be for the following access pattern: 0 1 2 3 7 0 3 8 1?
    1. M M M M M M H M M
    2. M M M M M H M M H
    3. M M M M M H H M M
    4. M M M M M H H M H
5. Select the correct average memory access time on a machine that takes 100 cycles to access DRAM, when you move from a 32kB cache (20% miss ratio, 1 cycle cache access latency) to a 2MB cache (5% miss ratio, 4 cycle cache access latency). The cache access latency is the time to lookup the cache, regardless of whether the cache is a hit or not.
    1. 32kB cache: 20 cycles, 2MB cache: 5 cycles
    2. 32kB cache: 20.8 cycles, 2MB cache: 8.8 cycles
    3. 32kB cache: 21 cycles, 2MB cache: 9 cycles
    4. Need more information
6. What data will be in a 4-entry, fully-associative, LRU cache with one word per line after the following memory accesses?
    address: 0, 1, 2, 3, 4, 5, 3, 2, 1
    1. 1, 2, 3, 4
    2. 1, 2, 3, 5
    3. 1, 3, 4, 5
    4. 1, 2, 4, 5

11. Virtual Memory

1. What are the benefits of virtual memory?
    1. It increases security by isolating each process' memory
    2. It frees the applications from having to manage a shared memory
    3. It allows an application to use more memory than physically available
    4. All of the above
2. Why would you not use larger pages?
    1. Each TLB access takes more time
    2. It can waste memory for programs that use small chunks of data
    3. It makes page faults to disk take longer
    4. B and C
3. What does the TLB do?
    1. It caches recently-used data
    2. It caches recently-used address translations
    3. It caches recently-used pages
    4. It caches recently-used instructions
4. How much data can a program access without a TLB miss on a machine with a 64 entry, 8-way set associative TLB where each page is 4kB, and a 16kB cache with a 64 byte line size.
    1. 16kB
    2. 256kB
    3. 1MB
    4. Don't have enough information
5. Why is a Physically Indexed and Virtually Tagged cache bad?
    1. Slow: have to do the translation before you can access the cache.
    2. Insecure: no way to prevent different programs from sharing data in the cache.
    3. Both A and B
    4. It's not: it is both fast and secure
6. What is correct about the pros and cons of multi-level and single-level page tables
    1. Single-level page tables provide quicker (shorter latency) translations
    2. Multi-level page tables consume more memory when the address space is sparsely populated, in other words very little of the virtual address space is used.
    3. Mult-level page tables consume less memory when the address space is totally used (all virtual pages have mappings).
    4. Multi-level page tables offer quicker (shorter latency) translations.

12. Parallelism

1. If I have a Intel Xeon Phi processor with 62 cores, what is the best speedup I could get from a program that is 99% parallelizable?
    1. 38.5
    2. 58.4
    3. 61.4
    4. 62.0
2. How do locks prevent problems with multiple processors accessing data at the same time?
    1. They tell the hardware to prevent multiple accesses to the data
    2. They tell the hardware to keep the data synchronized across multiple processors
    3. They tell the software when it's processor is allowed to access the data
    4. They allow the software to coordinate which processor accesses the data
3. An engineer decided to provide a lock for each processor to prevent them for fighting over locks. How well does that work? **`lk_n`** is provided to each processor.
```
if (lock(lk_n)) {
  load bal
  if (bal >= 500)
    store bal-500;
  else
    fail
} unlock(lk_n)
```
    1. Perfectly, the program achieves the desired performance improvement, without any incorrect execution
    2. The program does not achieve any performance change, and does not result in incorrect execution
    3. The program executes terribly slowly, but does not result in incorrect execution
    4. The program executes incorrectly.
4. We have two applications and systems. Which of the two performs the task quicker? Application A is able to run on 64 cores with 50% parallelizable. Application B is able to run on 4 cores with 90% of the application parallelizable.
    1. Application A will be sped up by 32x
    2. Application A will be sped up by 1.97x
    3. Application B will be sped up by 2.5x
    4. Application B will be sped up by 10x
5. What do you *need* to add to the pipeline to support executing load/store instructions in parallel with other non LD/ST instructions?
    1. Extra data memory
    2. Extra data memory ports
    3. Extra register file ports
    4. All of the above

6. The cache for processor 2 wants to load an address A. What needs to be true about the data in processor 1's cache such that processor 1 must send its data to processor 2 to avoid processor 2 getting incorrect data
    1. It has the line (tag match and valid)
    2. It has the line (tag match and valid) and it is not dirty
    3. It has the line (tag match and valid) and it is dirty
    4. It has the dirty line (tag and dirty)